Dynamics Queue Admin

Technical Design Document / Low-Level Design — dispatch-queue management console (`queadmin-frontend`, SPA), version 1.0.0

Document	**TDD/LLD-1.0.0** — Technical Design Document / Low-Level Design
Application	Dynamics Queue Admin (*Queue Admin*) — dispatch-queue management console of the Dynamics freight marketplace
Asset type	Web Application
Asset code	**ASSET-DYN-QA-001**
Standard	INSA Secure Website Management Standard, Version 1.0, 2014 EC
Platform	Vite 8 + React 19 + TypeScript SPA (React Router 7), Redux Toolkit / RTK Query, socket.io-client 4, hand-written CSS
Production host	`queue.dynamicsroute.tech`
Design version	1.0.0 (as-built)
Companion documents	SRS, System & Architecture Design (SDD), User Documentation (User Guide), Security Documentation
Submitting entity	Dynamics Transport
Contact person	Marew Masresha, General Manager — `mmarew@gmail.com`, `0983222221`
Prepared by	Marew Masresha, General Manager
Reviewed by	Marew Masresha, General Manager
Signatories	Marew Masresha, General Manager
Date of preparation	15 October 2026
Date of submission	15 October 2026

As-built document

This document describes the low-level design of the submitted release of the Dynamics Queue Admin console as implemented in `queadmin-frontend`. It is the technical counterpart to the SRS and the companion SDD, and is prepared for INSA Secure Website Management Standard review. Where behaviour is enforced by the backend service (`transportBackEndNative`) it is stated explicitly, so reviewers can distinguish client responsibility from backend authority. Every control named here resolves to a concrete file and symbol in the release tree.

Contents

1	Introduction
2	Architecture-to-code map
3	Build, tooling and configuration
4	Authentication and session management
5	Routing and authorization
6	State management
7	API integration
8	Realtime integration
9	Feature-level LLD
10	Data model and persistence
11	Status and state models
12	Validation, error handling and logging
13	Security implementation
14	Testing strategy
15	Control-to-code traceability
16	Deployment and operations
A	Appendix A — file/module index
B	Appendix B — document control

List of figures

Fig 2.1	Level-0 system context for the Queue Admin console
Fig 2.2	Level-2 logical architecture layers

> **How to read this document**
>
> Paths such as `src/lib/redux/api/base.ts` are relative to the repository root of the `queadmin-frontend` SPA. Identifier strings such as `FR-QUEUE`, `FR-BID` and `NFR-SEC` refer to requirement families defined in the companion `docs/insa/SDD.html` (Section 12) and the SRS. Controls `C1`–`C36` are introduced by Section 15. Requirement families are referenced only where they exist in that companion set; solely cross-cutting or build-time controls carry a dash (—) in the “derived from” column.

1. Introduction

1.1 Purpose

This Technical Design Document / Low-Level Design (TDD/LLD) specifies *how* the Dynamics Queue Admin console is built, at a granularity sufficient for an independent engineer to locate, understand and modify each behaviour. It complements the SRS (which states *what* the system must do) and the SDD (which states the system's structure and architectural decisions). Where the SDD names a component, this document names the file, the symbol and the control that implements it.

1.2 Scope

The scope is the `queadmin-frontend` single-page application (SPA) as built and submitted: the browser-side console used by queue organization administrators (role 11), queue dispatchers (role 12), transport-company administrators (role 3) and transport-company staff (role 6) to manage dispatch queues, organizations, members, shipper requests, bids and reports. The backend service `transportBackEndNative` is out of build scope but is referenced wherever it is the authoritative enforcement point; those references are labelled *Server* or *backend*.

Mobile clients, the transportCompany SPA, and infrastructure provisioning are out of scope except as external actors of the console.

1.3 Audience

This document is written for (a) INSA evaluation reviewers assessing conformity to the Secure Website Management Standard, (b) maintainers and reviewers of the `queadmin-frontend` codebase, and (c) security assessors who need to distinguish controls enforced in the browser from those enforced by the server. It assumes familiarity with React, TypeScript and HTTP.

1.4 Relationship to the SRS and SDD

The SRS requirement families are the traceability anchor. The SDD (Section 12) maps those families to design elements: `FR-AUTH`, `FR-ROLE/ACCESS`, `FR-ORG`, `FR-QUEUE`, `FR-BID`, `FR-ORDER`, `FR-REALTIME`, `FR-REPORT`, plus the non-functional families `NFR-SEC` and `NFR-DEPLOY`. Section 9 of this document drills each family down to the component and handler level, and Section 15 maps derived controls to file-and-symbol evidence. No requirement family is introduced here that is not present in the companion set.

1.5 Design principles and constraints

- **Server-authoritative security.** The client enforces role and ownership rules for usability and defence in depth, but the backend is the authority. Client guards are never presented as the security boundary (see Section 13.3).
- **Cookie session, no token in JavaScript.** The session JWT lives in an httpOnly cookie; JavaScript stores hold only a non-sensitive profile and no bearer token (Section 4.2).
- **Fail closed.** Unknown or unauthenticated route states redirect to login rather than rendering a protected view (Section 5.2).
- **Defensive decoding.** All backend payloads pass through decoders and schemas that tolerate absence and reject malformed values, so one bad record cannot crash a list (Section 7.3, Section 12.1).

- **Honesty over masquerade.** Settings controls with no backend support are disabled rather than faked, and known gaps are recorded as open findings (Section 11.5, Section 15.3).
- **Build-gated quality.** Type-check, lint and the unit suite are the release gates described in Section 3 and Section 14.

1.6 Conventions

Code paths are shown relative to the repository root. Line references (`file.ts:NN`) identify the as-built release and may shift by a few lines under reformatting; the enclosing symbol name is the durable reference. Controls are labelled MET, OPEN, BY DESIGN or **Server**/**Debt** as defined in Section 15.1.

> **Open finding carried into the design (C3)**
>
> The API contract comment and maintenance rule state that login and OTP verification send *no* `roleId`, so the backend resolves the account's own role and a role 12 dispatcher can sign in. The as-built service still transmits `roleId: 11` on both calls. This is recorded as an open finding (Section 4.1, C3) and is the only intentional client/contract divergence in the release.

2. Architecture-to-code map

This section connects the architectural view of the SDD to concrete source locations. It presents the system context (Level 0), the logical layers (Level 2) and the runtime sequence that wires them together at boot.

2.1 System context (Level 0)

The console is a browser SPA that talks to a single backend service over HTTPS (REST) and WSS (Socket.IO). It authenticates via an httpOnly cookie issued by the backend and reads or mutates only backend-owned data. It has no direct database access and no third-party runtime integrations.

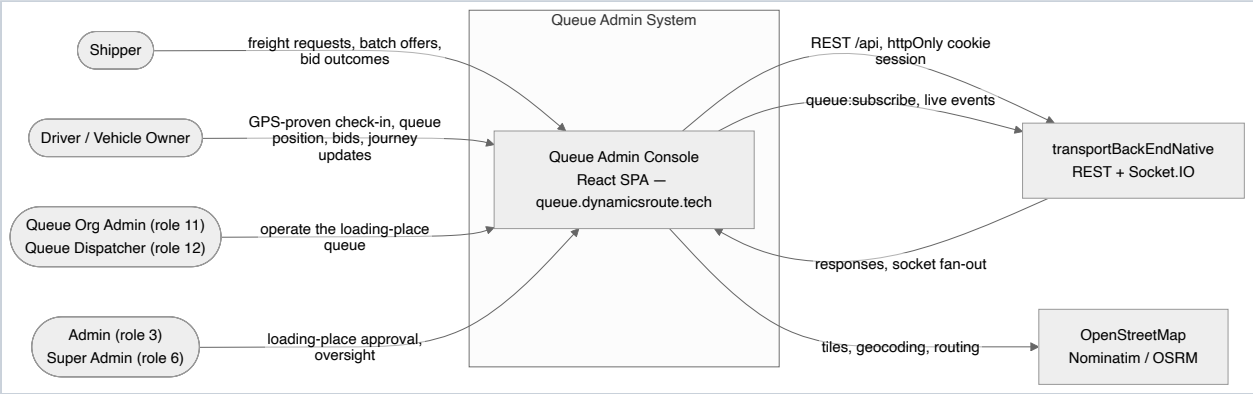


Figure 2.1 — Level-0 context: the SPA, its backend service, and the operator roles.

External actor / system	Direction	Channel	Purpose
Queue org admin (role 11) / dispatcher (role 12)	In	Browser	Manage queue organizations, dispatch queue, members, bids.
Transport admin (role 3) / staff (role 6)	In	Browser	Cross-organization administration and oversight.
`transportBackEndNative` REST API	In / out	HTTPS (JSON)	All data reads and mutations; session issuance and authority.
Socket.IO gateway	In	WSS	Queue and bid realtime events; cookie-authenticated handshake.

2.2 Logical layers (Level 2)

The application is layered so that presentation, orchestration and transport concerns are separable. The map below names each layer, its responsibility, and the directories that implement it.

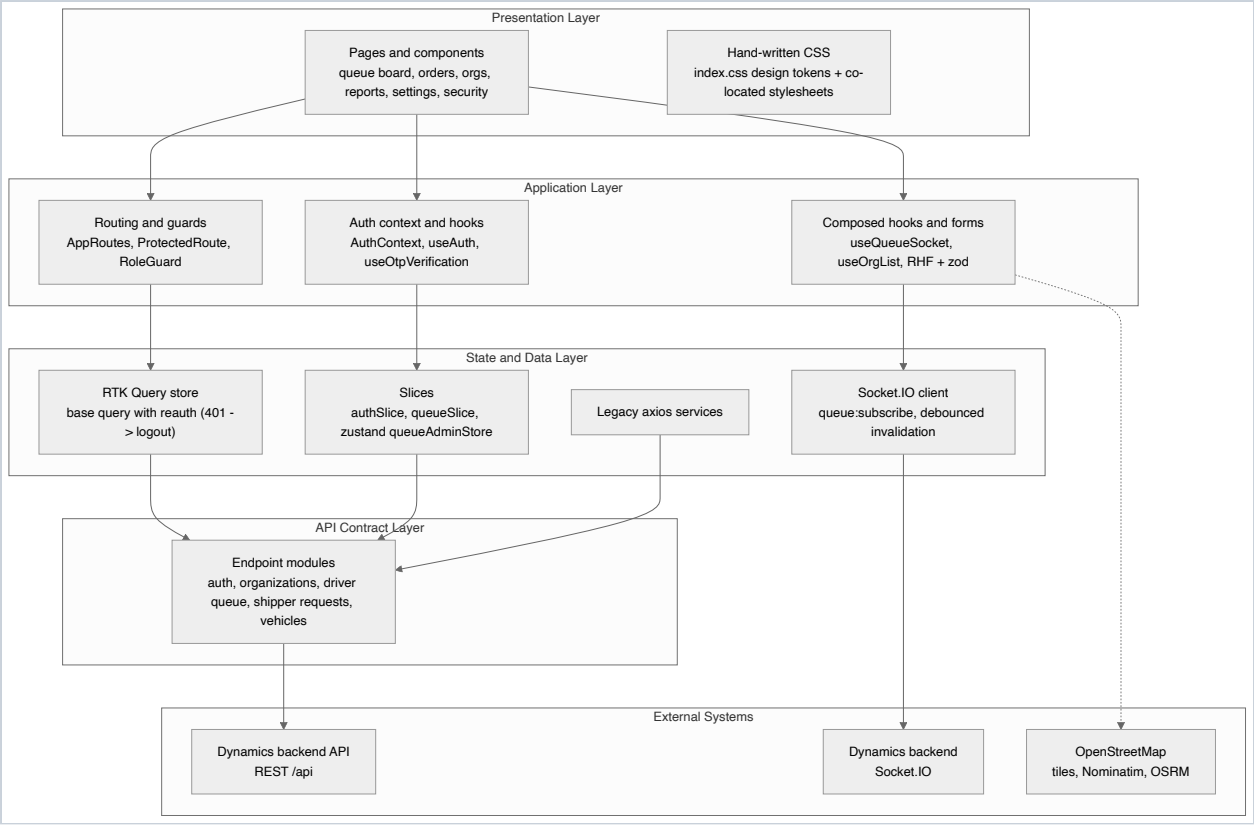


Figure 2.2 — Level-2 logical architecture layers.

Layer	Responsibility	Code locations
Routing & guards	Route composition, lazy loading, auth/role gates, org-presence gate.	`src/App.tsx`, `src/routes/AppRoutes.tsx`, `src/components/auth/ProtectedRoute.tsx`, `src/components/auth/RoleGuard.tsx`
Pages / features	Feature screens composing components and hooks.	`src/pages/**`, `src/components/**`
Orchestration hooks	Query/mutation wiring, socket subscription, derived metrics.	`src/hooks/**`, `src/components/**/<feature>.ts·tsx`
State	Global auth slice, UI/queue slice, persisted selection store.	`src/lib/redux/store.ts`, `src/lib/redux/slices/authSlice.ts`, `src/lib/redux/slices/queueSlice.ts`, `src/store/queueAdminStore.ts`
Transport / API	RTK Query API + endpoint modules, base query, reauth, tag invalidation.	`src/lib/redux/api/base.ts`, `src/lib/redux/api/*Endpoints.ts`, `src/lib/api.ts`
Realtime	Socket lifecycle, room subscription, debounced invalidation.	`src/lib/socket.ts`, `src/hooks/useQueueSocket.ts`
Domain / utils	Types, zod schemas, formatters, mappers, static INSA data.	`src/types/**`, `src/schemas/**`, `src/utils/**`, `src/data/insaStandard.ts`

2.3 Runtime narrative

The console reaches a usable state in a fixed sequence:

1. **Boot.** `src/main.tsx` mounts the React root, installs the Redux `Provider`, the i18n provider and the `AuthProvider`, then renders `App`.
2. **Session probe.** `AuthContext` reads the non-sensitive profile from `lib/auth.ts` storage; an absent profile yields the unauthenticated state and route guards redirect to login.
3. **Route resolution.** `AppRoutes` lazily loads the matched page behind `ProtectedRoute` and, where required, `RoleGuard`.
4. **Data hydration.** RTK Query hooks fire; the request flows through `baseQueryWithReauth` with `credentials: "include"`, so the httpOnly cookie accompanies every call.
5. **Realtime attach.** On authenticated mount, `useQueueSocket` ensures a single Socket.IO connection, subscribes the active organization, and translates events into debounced cache invalidation.
6. **Failure paths.** A 401 from REST or a 401 handshake refusal ends the session and clears local state (Section 4.3).

3. Build, tooling and configuration

3.1 Package manifest

The manifest is `package.json`. The runtime engine is Node `>=20.19.0`. The scripts that constitute the build and quality gates are:

Script	Command	Role
`build`	`tsc -b && vite build`	Type-check project references, then produce the production bundle.
`dev`	`vite`	Local dev server on port 5173 with API/socket proxy.
`lint`	`eslint . --ext ts,tsx --max-warnings 0`	Static lint gate (zero warnings tolerated).
`test`	`vitest run`	Unit test suite (Section 14).
`preview`	`vite preview`	Serve the built bundle locally.

Key runtime dependencies: `react` / `react-dom` 19, `react-router-dom` 7, `@reduxjs/toolkit` 2 + `react-redux` 9, `socket.io-client` 4, `zustand` 5, `zod` 4, `react-hook-form` + `@hookform/resolvers`, `axios`, `i18next` / `react-i18next`, `date-fns`, `lucide-react`, `sonner`. One declared dependency, `@tanstack/react-query`, is unused and is recorded as open debt (Section 11.5).

3.2 Vite configuration

`vite.config.ts` is a function config that loads environment variables and derives the dev proxy target. Design points:

- **Path alias.** `@` resolves to `./src` via `path.resolve(import.meta.dirname, "./src")`, keeping imports stable.
- **Manual chunking.** `build.rollupOptions.output.manualChunks` splits `vendor` (React, React-DOM, React-Router), `redux` (Redux Toolkit, React-Redux, Zustand) and `icons` (lucide-react) from application code, improving cacheability.
- **Source maps off in production.** `sourcemap: process.env.NODE_ENV !== "production"` emits source maps only outside production, so shipped bundles do not expose original sources (C27).
- **Dev proxy.** When `VITE_API_BASE_URL` is set, `/api` is proxied with `changeOrigin` and `/socket.io` additionally with `ws: true`, so cookies and websockets work in development without CORS gymnastics.
- **Test include.** Vitest matches `src/**/*.{test,spec}.?(c|m)[jt]s?(x)`.

`@vitejs/plugin-react` is the sole build plugin. There is no custom CSP injection plugin in this release; Content-Security-Policy and related HTTP security headers are a server/hosting responsibility and are recorded as deferred (C36).

3.3 TypeScript

The project uses TypeScript project references; `tsc -b` is the first half of `build` and thus a hard gate (C28). Types are shared through `src/types/**`, with API request/response shapes and role constants colocated in `src/types/queue.ts`. Domain schemas are authored with `zod` in `src/schemas/queue.ts` and consumed by `react-hook-form` resolvers.

3.4 Environment and base-URL resolution

Environment variables are read at build time by Vite (they are compiled into the bundle, so changing them requires a rebuild). Resolution is centralised in `src/utils/baseUrl.ts`:

Concern	Resolution order
REST base URL	`VITE_API_BASE_URL` → `VITE_API_URL` → `"/api"`; normalised so the result ends in `/api`.
Socket URL	`VITE_WEBSOCKET_URL` → `VITE_SOCKET_URL` → `VITE_API_BASE_URL` → `window.location.origin`.

This resolves the `NFR-DEPLOY` concern that the same artifact can be pointed at different environments without code edits, and documents the constraint that the values are bake-in-at-build.

3.5 Build and static gates summary

Gate	Command	Control
Type check	`tsc -b`	C28
Lint	`npm run lint`	C29
Unit tests	`npm test`	C30
Production bundle	`vite build`	C26, C27

4. Authentication and session management

4.1 Flow

Authentication is passwordless OTP. The console never stores a password and never receives or stores the JWT. The flow, as built:

1. The user opens `/login` (`Auth` with `initialStep="login"`) and submits a phone number. `Auth` calls `requestLoginOtp()` (`src/services/auth.service.ts`), a `POST` to `/user/loginUser`.
2. The backend sends an OTP out of band. The console renders the OTP entry field (`useOtpVerification`, `OTP_LENGTH = 6`) with a 60-second (`RESEND_COOLDOWN_SECONDS`) resend cooldown; it auto-submits when six digits are complete.
3. `verifyOtp(phoneNumber, otp)` `POST`s to `/user/verifyUserByOTP`. On success the backend issues the session JWT as an httpOnly cookie (Section 4.2) and returns the non-secret `userData`.
4. The client calls `setAuth({ userData })` (`AuthContext`), which persists the profile and dispatches the Redux `setAuth` action; the socket is connected from the `AuthContext` effect.
5. The user is navigated to the originally requested route (or `/dashboard`).

> **Open finding C3 — role-omitted login contract vs. as-built body**
>
> `src/utils/constant.ts:16–18` states, as a maintenance rule, that login and OTP verification send *no* `roleId` so the backend resolves the account's own role from `UserRole`. That is the intended contract and the reason the console can admit both role 11 and role 12. However, the as-built `requestLoginOtp` and `verifyOtp` in `src/services/auth.service.ts:19,27` still transmit `roleId: QUEUE_ORG_ADMIN_ROLE` (11), contradicting their own docblock (`lines 8–15`) which repeats the role-omitted rule. Registration (`registerUser`) correctly sends 11 because provisioning *is* the role being created. This divergence is tracked as C3 and is the only known client/contract mismatch; it does not weaken the session model but can cause role-12 login rejection if the backend honours the asserted role.

4.2 Where the session lives

The session credential is an httpOnly cookie named `token`, set by the backend (`Utils/AuthCookie.js`) when the OTP is verified. Because it is httpOnly, no JavaScript — including this console — can read it. The client carries it automatically because every request is configured `withCredentials` / `credentials: "include"` (`src/lib/api.ts:9–12`, `src/lib/redux/api/base.ts:38–41`).

JavaScript storage holds only the non-secret profile, under `queueadmin:user` (`src/lib/auth.ts`). The record is `{ userData: { userId, userUniqueId, fullName, phoneNumber, email, roleId } }` (`StoredUser`). A one-time migration (`purgeLegacyAuth`) rewrites any legacy `queueadmin:auth` record to the profile-only shape and removes the legacy keys (`localStorage` and `sessionStorage`); see C6.

Artifact	Location	Contains	Readable by JS
Session JWT	`token` httpOnly cookie	Signed session claims	No
User profile	`localStorage["queueadmin:user"]`	Non-secret profile only	Yes

Artifact	Location	Contains	Readable by JS
Queue selection	`localStorage["queueadmin:store"]` (zustand persist)	`selectedOrgId` only (partialized)	Yes

4.3 Logout

Logout is centralised and defensive. `AuthContext.logout()` (`src/context/AuthContext.tsx:38–44`) runs three steps in order: disconnect the socket, fire a best-effort `POST /user/logout` to clear the server cookie, then clear local profile storage and dispatch the Redux `logout` action — which sets `auth` to `null` and always clears local storage (C9). Local state is cleared unconditionally even if the server call fails, so a network error cannot strand the user in a half-authenticated state.

Two additional paths end the session:

- **Global REST 401** — `baseQueryWithReauth` (`src/lib/redux/api/base.ts:61–67`) intercepts any 401 and dispatches `logout()`; the axios transport does the same in `src/lib/api.ts:15–26` (clear storage and redirect to `/login`). Because the auth slice is the single source of truth, every authed page unmounts at once (C7).
- **Socket 401 handshake** — `connect_error` whose message contains “unauthorized” or “token” disconnects and ends the session via a dynamically-imported dispatch, avoiding an endless reconnect loop (C8, `src/lib/socket.ts:168–178`).

4.4 Guards

`ProtectedRoute` (`src/components/auth/ProtectedRoute.tsx:16–60`) requires a stored profile and, by default, at least one queue organization. It: redirects to `/login` when unauthenticated; withholds render while the organization list is loading; redirects to `/setup-org` when no organization exists; and *fails open* on a query error (`lines 48–52`) so a transient list error does not lock the user out. `RoleGuard` (`src/components/auth/RoleGuard.tsx:14–27`) admits the allow-list `{11, 12, 3, 6}` and redirects anything else to `/login` (C1, C2). Both guards are convenience layers; the backend is the authority (Section 13.3).

4.5 Session lifetime

Cookie and JWT lifetimes are role-based and server-set (`Utils/SessionPolicy.js`, `Utils/AuthCookie.js`): system admins (roles 3 and 6) receive `SESSION_TTL_ADMINS` (default 24h); all other roles — including queue org admins (11) and dispatchers (12) — receive `SESSION_TTL_OTHERS` (default 365d). The cookie lifetime matches the JWT lifetime it carries. The console has *no refresh-token flow*: any expiry surfaces as a 401, which triggers logout, so re-authentication means signing in again.

5. Routing and authorization

5.1 Route table

Routes are declared once in `src/routes/AppRoutes.tsx` inside a single `<Suspense>` boundary; every page is code-split with `React.lazy` (C25). The table lists the as-built route behaviour:

Path	Element	Guards	Notes
`/login`	`Auth (initialStep="login")`	None	Public.
`/register`	`Auth (initialStep="register")`	None	Public; provisions role 11.
`/setup-org`	`SetupOrganization`	`ProtectedRoute requireOrg={false}`	Requires login, not an existing org.
`/organizations`	Redirect → `/dashboard`	—	Standalone page unreachable (open item, Section 11.5).
`/dashboard`	`QueueDashboardPage`	`ProtectedRoute` + `RoleGuard`	Live dispatch queue.
`/orgs/:queueOrganizationUniqueId`	`QueueOrgManagePage`	`ProtectedRoute` + `RoleGuard`	Organization administration.
`/reports`	`ReportsPage`	`ProtectedRoute` + `RoleGuard`	KPIs and analytics.
`/orders`	`OrdersPage`	`ProtectedRoute` + `RoleGuard`	Shipper requests / batches.
`/orgs/:queueOrganizationUniqueId/orders`	`OrdersPage`	`ProtectedRoute` + `RoleGuard`	Org-scoped orders view.
`/security`	`SecurityPage`	`ProtectedRoute` + `RoleGuard`	INSA compliance self-assessment.
`/settings`	`SettingsPage`	`ProtectedRoute` + `RoleGuard`	Theme / language / display only.
`/`	Redirect → `/dashboard`	—	Landing redirect.
`*`	Redirect → `/organizations` → `/dashboard`	—	Catch-all fallback.

5.2 Guard stack and fail-closed behaviour

Protected routes compose two guards: `ProtectedRoute` (authentication + organization presence) on the outside, `RoleGuard` on the inside. The composition is fail-closed: an unauthenticated request is redirected to `/login` before any protected component mounts; an authenticated user without an organization is redirected to `/setup-org`; and a caller outside the role allow-list is redirected to `/login` (C1, C2). The single intentional permissive branch is the organization-query error path, which fails open to avoid a lock-out, and is not a security boundary.

6. State management

State is split by lifetime and sensitivity: a Redux store for session and server cache, and a small persisted zustand store for the active organization selection. There is no global state for form input; forms are local React state or `react-hook-form`.

6.1 Redux store shape

`src/lib/redux/store.ts` configures the store with three reducers — `auth`, `queue` and the RTK Query `api` reducer — and appends `api.middleware` for cache/tag orchestration.

Slice	State	Actions
`auth`	`{ auth: StoredAuth | null; hydrated: boolean }`	`hydrateAuth`, `setAuth`, `logout`
`queue`	`{ selectedOrgId: string | null; orgsCache: QueueOrganization[] | null }`	`setSelectedOrgId`, `setOrgsCache`, `clearQueueState`
`api` (RTK Query)	Per-endpoint cache entries + tag registry	Generated hooks; invalidated by tag

6.2 Slice behaviours that matter

- **Auth is the single source of truth.** `AuthContext` reads `state.auth.auth` rather than owning state, so a 401 anywhere flips the UI to logged-out without a second source of truth (`AuthContext.tsx:20–23`). The slice initialises from `getStoredAuth()` so a reload rehydrates the profile synchronously. `setAuth` and `logout` in the slice also write/clear storage, keeping Redux and `localStorage` consistent (C6).
- **Queue slice is scratch state.** `selectedOrgId` and `orgsCache` support UI selection; they are derived from URL/query data and can be cleared without data loss.

6.3 Persisted UI store (zustand)

`src/store/queueAdminStore.ts` is a persisted zustand store under the key `queueadmin:store`. Only `selectedOrgId` is persisted via `partialize`; `socketConnected` is intentionally transient. A one-time cleanup deletes a stale `userCreatedOrgs` key from an earlier schema. Selecting the active organization therefore survives reloads without persisting any server or secret data.

6.4 RTK Query API configuration

`src/lib/redux/api/base.ts` defines one API object with `reducerPath: "api"`, `refetchOnFocus: false`, `refetchOnReconnect: false` and `keepUnusedDataFor: 300` (seconds). Disabling focus/reconnect refetch keeps realtime correctness with the socket-driven invalidation described in Section 8, avoiding redundant network churn. Nine tag types are registered: `QueueOrganizations`, `QueueOrgMembers`, `QueueStatus`, `DriverQueue`, `DriverBids`, `DriverDirectory`, `Auth`, `VehicleTypes` and `ShipperRequests`.

Endpoints are not declared on the base object; each feature module calls `api.injectEndpoints(...)` on the shared API, keeping features isolated while sharing one reducer/middleware. A stale comment at `base.ts:92` says “now 22 total”; the as-built total is **28** endpoints across five modules (auth 3,

queue-organization 11, driver-queue 9, shipper-request 3, vehicle 2). The comment is documentation debt only and is catalogued in Section 7.1.

7. API integration

7.1 Endpoint catalogue

The console issues **28** operations defined across five endpoint modules, all injected onto the single RTK Query API. Path fragments live in `src/utils/constant.ts` (the stated single source of truth). Custom `queryFn` endpoints are marked. Tag behaviour drives realtime cache coherence (Section 8).

Operation (hook)	Method	Path	Tag behaviour
`useRequestLoginOtpMutation`	POST	`/user/loginUser`	None (auth).
`useVerifyOtpMutation`	POST	`/user/verifyUserByOTP`	`onQueryStarted` persists profile via `storeAuth`; no cache tag.
`useRegisterUserMutation`	POST	`/user/createUser`	None.
`useListQueueOrganizationsQuery`	GET	`/queueOrganization` (`limit=100`)	provides `QueueOrganizations`.
`useGetQueueOrganizationQuery`	GET	`/queueOrganization/:id`	provides `{QueueOrganizations, id}`.
`useUpdateQueueOrganizationMutation`	PATCH	`/queueOrganization/:id`	invalidates `QueueOrganizations` + id.
`useApproveQueueOrganizationMutation`	PATCH	`/queueOrganization/:id/approve`	invalidates `QueueOrganizations` + id.
`useDeleteQueueOrganizationMutation`	DELETE	`/queueOrganization/:id`	invalidates `QueueOrganizations`.
`useCreateQueueOrganizationMutation`	POST	`/queueOrganization`	invalidates `QueueOrganizations`.
`useListQueueOrgMembersQuery`	GET	`/queueOrganization/:id/members`	provides `{QueueOrgMembers, id}`.
`useAddQueueOrgMemberMutation`	POST	`/queueOrganization/:id/members`	invalidates members + orgs; `userUniqueId` in body, not path.
`useDeactivateQueueOrgMemberMutation`	PATCH	`/queueOrganization/:id/members/:membershipId/deactivate`	invalidates members + orgs.
`useReactivateQueueOrgMemberMutation`	PATCH	`/queueOrganization/:id/members/:membershipId/reactivate`	invalidates members + orgs.
`useDeleteQueueOrgMemberMutation`	DELETE	`/queueOrganization/:id/members/:membershipId`	invalidates members + orgs.
`useGetQueueStatusQuery`	GET	`/queue/status` (`queueOrganizationUniqueId`, `queueDate`)	provides scoped `QueueStatus`, `LIST` and type tags.

Operation (hook)	Method	Path	Tag behaviour
`useManualCheckinMutation`	POST	`/queue/manualCheckin`	invalidates `QueueStatus:id|today`, `DriverQueue:id`.
`useDispatchQueueMutation`	POST	`/queue/dispatch`	invalidates queue tags + `ShipperRequests`.
`useAcceptDriverRequestMutation`	PUT	`/shipper/acceptDriverOffer`	custom `queryFn`; requires explicit ids; invalidates scoped queue + bids.
`useGetBidsForOrderQuery`	GET	`/queue/bidding/order/:shipperRequestUniqueId/bids`	provides `{DriverBids, id}`; `page/limit` params.
`useApproveBiddingMutation`	POST	`/queue/bidding/approve`	invalidates `ShipperRequests` (per-order gate).
`useOverrideEntryMutation`	PATCH	`/queue/entry/:queueUniqueId/override`	invalidates `QueueStatus`, `DriverQueue`.
`useRemoveEntryMutation`	DELETE	`/queue/entry/:queueUniqueId`	invalidates `QueueStatus`, `DriverQueue`.
`useGetEntryHistoryQuery`	GET	`/queue/entry/:queueUniqueId/history`	provides `{DriverQueue, <id>-history}`.
`useCreateQueueOrderMutation`	POST	`/shipperRequest/createRequest`	invalidates `QueueStatus`, `DriverQueue`, `ShipperRequests`.
`useGetShipperRequestsQuery`	GET	`/user/getShipperRequest4allOrSingleUser`	provides `ShipperRequests`; defaults `target=all&limit=100`.
`useGetShipperRequestBatchesQuery`	GET	`/shipperRequestBatch`	provides `ShipperRequests`; `requestMode=company_target`.
`useListVehicleTypesQuery`	GET	`/admin/vehicleTypes` (`limit=100`)	custom `queryFn`; falls back to defaults; provides `VehicleTypes`.
`useListVehicleDriversQuery`	GET	`/queue/driverDirectory`	provides `DriverDirectory`; `page=1&limit=20`.

7.2 Transport client

Two HTTP clients coexist. The RTK Query path uses `fetchBaseQuery` (`src/lib/redux/api/base.ts:38-41`) with `baseUrl: getBaseUrl()` and `credentials: "include"`, wrapped by `baseQueryWithReauth` which converts any 401 into a global `logout()`. The service path uses a shared axios instance (`src/lib/api.ts:9-26`) with `withCredentials: true` and its own 401 interceptor (clear storage + redirect to `/login`). Both automatically carry the httpOnly cookie, so neither adds an Authorization header. The active OTP login uses the axios service path; feature screens use the RTK Query hooks.

`getBaseUrl()` (`src/utils/baseUrl.ts`) resolves the REST base as described in Section 3.4 and normalises it to end in `/api`.

7.3 Defensive payload handling

- **Vehicle types.** `useListVehicleTypesQuery` is a `queryFn` that, on any error or malformed shape, registers and returns `DEFAULT_VEHICLE_TYPES`, so order creation is never blocked by missing reference data (C20, C31).
- **Explicit-id bid acceptance.** `useAcceptDriverRequestMutation` validates each id against a UUID pattern and, when an id is missing, returns a visible `CUSTOM_ERROR` instead of guessing. The former heuristic (fetch 100 requests and fuzzy-match) was removed because under concurrent bidding it could resolve to the wrong driver with no error (C18).
- **Formatters.** `src/utils/formatters.ts` and the order mappers decode optional fields defensively so a single malformed record degrades one row rather than crashing a list.

7.4 Idempotency and cache notes

Mutations invalidate by tag rather than patching cache entries (C22). List reads are deliberately bounded to keep payloads predictable: queue organizations and shipper requests use `limit=100`; batches use `limit=100` with `includeBids`; the driver directory uses `limit=20` (C35). There is no endpoint retry/backoff layer beyond the socket reconnection policy (Section 8.4).

8. Realtime integration

8.1 Socket lifecycle

`src/lib/socket.ts` owns a single module-scoped Socket.IO connection. It is created by `connectSocket()` (called from the `AuthContext` effect when an authenticated profile exists) and torn down by `disconnectSocket()` on logout. Connection options: transports `["websocket", "polling"]`, `withCredentials: true`, `auth` carrying `{ user, phoneNumber }`, reconnection enabled with 5 attempts, 1.5s–5s backoff, and a 20s timeout (C5, C8).

The socket identity is computed by `extractCredentials()`: roles 3 and 6 map to `"admin"`; every other role — including queue org admin (11) and dispatcher (12) — maps to `"queueOrgAdmin"`. Role 12 is deliberately *not* given its own identity: the backend keys sockets as `${userType}:${phone}` and queue fan-out selects `roleId IN (11,12)` under the `queueOrgAdmin` key, so a distinct dispatcher identity would silently receive no events (C24).

8.2 Event routing

Inbound payloads are handled by `handleQueuePayload()`, which tolerates both JSON strings and objects, extracts a message type from `messageTypes` (string or `{ type, message }`) or the `message` field, and ignores acknowledgements and heartbeats (`subscribed`, `unsubscribed`, `ping`, `pong`). Non-acknowledgement events are forwarded to registered handlers (`onQueueEvent`) and simultaneously trigger a debounced cache invalidation (C23).

Signal	Classification	Invalidation (debounced)
`queue_org_approved`, `queue_org_updated`, `queue_org_deleted`, `queue_member_added`, `org_approved`, or a payload with org id + approval status	Organization event	`QueueOrganizations` after 100 ms.
Any other queue/bid event	Operational event	`QueueStatus`, `DriverQueue`, `ShipperRequests` after 250 ms.

The debounce collapses bursts into one invalidation, and a parse failure still triggers an invalidation so the cache cannot drift permanently stale (C23).

8.3 Subscription model

Subscriptions are refcounted in a module-scoped `Map` keyed `${queueOrganizationUniqueId}_${queueDate||""}`. `subscribeToQueue()` increments the refcount (and connects if needed); `unsubscribeFromQueue()` decrements and only emits `queue:unsubscribe` when the count reaches zero (C24). On every (re)connect the handler replays all active subscriptions with `queue:subscribe`, so a transient network blip does not silently unsubscribe the page (`src/lib/socket.ts:156–161`).

8.4 Connection state and revocation

Connectivity is mirrored into the zustand store via `setSocketConnected` and surfaced by `useQueueSocket` as `{ socketConnected, isLive }`. A `connect_error` whose message contains

“unauthorized” or “token” disconnects and dispatches the Redux `logout`, ending the session rather than retrying indefinitely (C8, `src/lib/socket.ts:168–178`). The dynamic imports of the store and slice (`clearSessionOnUnauthorizedSocket`) avoid a module cycle.

9. Feature-level LLD

This section drills each requirement family from the companion SRS/SDD down to the components, handlers and schemas that implement it: `FR-AUTH`, `FR-ROLE/ACCESS`, `FR-ORG`, `FR-QUEUE`, `FR-BID`, `FR-ORDER`, `FR-REALTIME` and `FR-REPORT`, with the cross-cutting `NFR-SEC` and `NFR-DEPLOY` families realised as the numbered controls of Section 15.

9.1 Authentication (FR-AUTH)

Authentication is phone-first and passwordless. Both `/login` and `/register` render `src/components/auth/Auth.tsx` with an `initialStep` prop (`src/routes/AppRoutes.tsx:58-59`); `Auth` is the step controller that mounts `Login.tsx`, `VerifyOtp.tsx` and `Register.tsx` inside `AuthLayout` / `AuthShell`. Shared inputs live in `fields.tsx`; the six-box one-time-code entry is `OtpDigitInputs.tsx`.

Step	Component / symbol	Behaviour
Phone capture	`Login.tsx`, `fields.tsx`, `loginSchema`	Phone entered as `X-XX-XX-XX-XX`, validated and normalised by `utils/phoneFormatter.ts` (C19).
Login request	`services/auth.service.ts → login()`	`POST user/loginUser` with `{ phoneNumber, roleId: 11 }`; the backend issues a one-time code (C3, open finding F1).
Code verification	`VerifyOtp.tsx`, `auth.service.ts → verifyOtp()`	`POST user/verifyUserByOTP`; on success the backend sets the httpOnly session cookie and returns a non-sensitive profile (C11, C4).
Registration	`Register.tsx`, `registerSchema`	`POST user/createUser` with sanitised `fullName`, `phoneNumber` and optional `email` (C16, C17).
Persist profile	`context/AuthContext.tsx`, `lib/auth.ts`	Only the profile is written to `queueadmin:user`; no bearer token is stored (C11, C12).
Logout	`AuthContext.tsx → logout()`	Best-effort `POST user/logout`, then `clearAuth()` and the Redux `logout` action (C15).

Auth HTTP is performed by the shared axios instance (`lib/api.ts`), so a 401 from any call clears the session (C14, Section 13.3). There is no password field and no token refresh flow: a session that expires forces a return to `/login` (open finding F6, Section 15.3).

9.2 Roles and access control (FR-ROLE/ACCESS)

Two wrappers, composed in `src/routes/AppRoutes.tsx`, guard the application. `ProtectedRoute` enforces authentication (and, unless `requireOrg={false}`, the existence of an organisation, redirecting to `/setup-org` otherwise); `RoleGuard` enforces the role allow-list `{11, 12, 3, 6}` and redirects disallowed roles to `/login` (C6, C7, C9). Route components are lazy-loaded under a single `Suspense` fallback (`PageLoader`).

Path	Element	Guard
`/login`, `/register`	`Auth`	Public

Path	Element	Guard
`/setup-org`	`SetupOrganization`	`ProtectedRoute requireOrg={false}`
`/dashboard`	`QueueDashboardPage`	`ProtectedRoute` + `RoleGuard`
`/orgs/:queueOrganizationUniqueId`	`QueueOrgManagePage`	`ProtectedRoute` + `RoleGuard`
`/orders`, `/orgs/:id/orders`	`OrdersPage`	`ProtectedRoute` + `RoleGuard`
`/reports`	`ReportsPage`	`ProtectedRoute` + `RoleGuard`
`/security`	`SecurityPage`	`ProtectedRoute` + `RoleGuard`
`/settings`	`SettingsPage`	`ProtectedRoute` + `RoleGuard`
`/organizations`, `/`, `*`	Redirects	to `/dashboard` or `/organizations`

The allow-list is a usability gate; the backend re-authorises every action. Client checks are documented as defence in depth, not as the security boundary (C9).

9.3 Organisations and membership (FR-ORG)

- **Setup.** `components/organization/SetupOrganization.tsx` collects name, type, phone, address and coordinates, validated by `setupOrgSchema`, and creates the organisation through the `queueOrganization` endpoints.
- **Management.** `pages/queue-org-manage/QueueOrgManagePage.tsx` renders an organisation's profile and members; the child parts are `OrgProfileForm.tsx` (`queueOrgProfileSchema`), `AdminApprovalActions.tsx` (approve) and `StatusBadge.tsx`.
- **Membership.** `MembersTable.tsx` gates add/deactivate/reactivate/remove behind `canManage` (role 11) and calls `queueOrganization/:id/members` with the member's `userUniqueId` in the request body (C10).
- **Active organisation.** The selected organisation is held in the Redux `queue` slice (`selectedOrgId`) and mirrored in `store/queueAdminStore.ts`, so tenancy follows the operator's selection (C10).

`pages/organizations/OrganizationsPage.tsx` is present but not routed; only its stylesheet is imported by the dashboard. It is recorded as a legacy/unused module in Appendix A.

9.4 Queue operations (FR-QUEUE)

Operation	Components	Schema / endpoint
Board and metrics	`pages/dashboard/QueueDashboardPage.tsx`, `queue/QueueBoard.tsx`, `queue/table/QueueDesktopTable.tsx`, `queue/table/QueueMobileTable.tsx`, `queue/QueueVehicleTypeCard.tsx`	`queue/status`

Operation	Components	Schema / endpoint
Dispatch	`queue/DispatchModal.tsx + queue/dispatch/*`	`dispatchSchema` → `queue/dispatch`
Manual check-in	`queue/CheckinModal.tsx`, `queue/checkin/useCheckinData.ts`, `checkin/CheckinDriverSearch.tsx`, `checkin/CheckinPositionSection.tsx`	`checkinSchema` → `queue/manualCheckin`
Queue override	`queue/OverrideModal.tsx`	`overrideSchema` → `queue/entry/:queueUniqueId/override`
Entry history	`queue/EntryHistoryModal.tsx`	`GET queue/entry/:queueUniqueId/history`
Create order	`queue/CreateOrderModal.tsx + queue/order-modal/*`	`createOrderSchema` → `shipperRequest/createRequest`, `shipperRequestBatch`

Check-in note. `checkinSchema` requires only the driver `vehicleDriverUniqueId` (a UUID) and an optional positive integer `queueNumber`. When no number is supplied, the console shows an auto-assigned position estimate (`CheckinPositionSection`). There is *no* client-side geofence gate: a modelled `checkinRadiusKm` value is not enforced, and `calculateDistanceKm` is used only to display order distances (Section 11.5).

9.5 Bidding (FR-BID)

Driver bids for a shipper request are shown by `components/orders/DriverBidsModal.tsx` and the `orders/bids/*` family (`BidsToolbar`, `BidRow`, `BidOfferComparison`, `BiddingBoardToggle`). Listing and filtering are handled by `useDriverBidsFilter.ts` over `GET queue/bidding/order/:shipperRequestUniqueId/bids`; rows are normalised by `mapDriverBids.ts` (`mapDriverBidToRow`, `isBidActionable`). Acceptance is performed by `useAcceptDriverBid.ts`, which submits explicit driver/user identifiers to `queue/bidding/approve` (and the shipper-side `shipper/acceptDriverOffer`). The handler accepts *only* the named identifiers and fails loudly otherwise — there is no silent auto-fallback (C10).

9.6 Orders and shipper requests (FR-ORDER)

`pages/orders/OrdersPage.tsx` and `pages/orders/ordersDataMapper.ts` drive the order list, supported by `OrdersTable`, `OrdersCardsList`, `OrdersModals`, the `mobile/*` cards and the `bids/*` sub-family. Route display helpers `trimAddress`, `formatTrimmedRoute` and `formatRoute` live in `orders/OrdersTypes.ts`; `extractOfferCost` and `calculateDistanceKm` in `utils/formatters.ts`. New requests are created from the order modal via `shipperRequest/createRequest` and `shipperRequestBatch`.

9.7 Realtime (FR-REALTIME)

The Socket.IO client and its lifecycle are described in Section 8. `hooks/useQueueSocket.ts` is the page-facing consumer, exposing `{ socketConnected, isLive }`, subscribing the active queue, and translating inbound events into the debounced cache invalidations of Section 8.2 (C23, C24).

9.8 Reports (FR-REPORT)

`pages/reports/ReportsPage.tsx` presents operator metrics computed by `hooks/useReportsMetrics.ts` from the queue and order queries. All figures are derived from server state; nothing is persisted client-side (C30).

9.9 Security self-assessment (NFR-SEC, INSA)

`pages/security/SecurityPage.tsx` renders four tabs — overview, requirements, vulnerabilities and testing — from the embedded INSA standard data in `src/data/insaStandard.ts` (5 focus areas, 35 requirements, Annex A vulnerabilities 1–8, Annex B tests 1–102). Progress is tracked by `hooks/useSecurityChecklist.ts` and `utils/securityChecklist.ts` over the local key `insa_security_checklist_v1`; `SecurityChecklist.tsx` adds search and status filtering (C36).

10. Data model and persistence

10.1 Client-side domain types

Domain shapes are declared once in `src/types/queue.ts` (roles, organisation types `QUEUE_ORG_TYPES`, journey status, queue and member records) and consumed by the RTK Query layer. Argument and result shapes for the API hooks are declared in `src/lib/redux/api/types.ts`. Because the backend is authoritative for business objects, the client types are deliberately tolerant of absent optional fields rather than asserting server data into existence (Section 7.3, Section 12.1).

10.2 Client persistence map

The console persists only a small, non-sensitive set of values. No bearer token and no business data are written to browser storage; the session itself lives in an httpOnly cookie the JavaScript cannot read (C11, C28).

Key	Storage	Written by	Contents	Sensitivity
`queueadmin:user`	localStorage (sessionStorage fallback)	`lib/auth.ts`	Non-sensitive profile (no token)	Low
`queueadmin:auth` *(legacy)*	localStorage	Purged on load by `purgeLegacyAuth`	Old JWT — removed if present	was High → eliminated (C12)
`queueadmin:store`	localStorage (zustand persist)	`store/queueAdminStore.ts`	`selectedOrgId` only (partialised)	Low
`app_dark_mode`	localStorage	`contexts/ThemeContext.tsx`	`"true"` / `"false"`	None
`app_language`	localStorage	`i18n/index.ts`	`"en"` / `"am"`	None
`insa_security_checklist_v1`	localStorage	`hooks/useSecurityChecklist.ts`	Per-test pass/fail map	None
Session cookie *(server)*	httpOnly cookie	Backend at `verifyUserByOTP`	Session JWT	High — not readable by JS (C11)

10.3 Server-authoritative data

Organisations, queues, members, bids, shipper requests and vehicle types are held by the backend service. The client keeps them only as in-memory RTK Query caches (Section 10.4) and re-reads them from the server; it never treats a cached copy as authoritative (C30).

10.4 RTK Query cache

The API slice (`src/lib/redux/api/base.ts`) defines nine cache tags: `QueueOrganizations`, `QueueOrgMembers`, `QueueStatus`, `DriverQueue`, `DriverBids`, `DriverDirectory`, `Auth`, `VehicleTypes` and `ShipperRequests`. Mutations invalidate the tags they affect, and realtime events invalidate the same tags through the debounced path of Section 8.2, so the cache converges on the server's view rather than on pushed payloads (C23, C30).

11. Status and state models

11.1 Journey / order status

Journey status is an integer code rendered through the map in `src/utils/journeyStatus.ts`. Codes 1–20 are the closed vocabulary; an unrecognised code falls back to a neutral label rather than throwing.

Code	Symbol	Meaning
1	`waiting`	Queued, not yet requested
2	`requested`	Offer requested of a driver
3	`acceptedByDriver`	Driver accepted
4	`acceptedByShipper`	Shipper accepted
5	`goToLoadingPlace`	Driver en route to load
6	`loading`	Loading in progress
7	`loaded`	Loaded
8	`journeyStarted`	Journey started
9	`journeyCompleted`	Journey completed
10	`cancelledByShipper`	Cancelled by shipper
11	`rejectedByShipper`	Rejected by shipper
12	`cancelledByDriver`	Cancelled by driver
13	`cancelledByAdmin`	Cancelled by admin
14	`completedByAdmin`	Completed by admin
15	`cancelledBySystem`	Cancelled by system
16	`noAnswerFromDriver`	Driver did not answer
17	`notSelectedInBid`	Bid not selected
18	`rejectedByDriver`	Rejected by driver
19	`replacedByCompanyAssignment`	Replaced by company assignment
20	`partiallyCancelled`	Partially cancelled

11.2 Organisation approval status

Organisations carry an approval state rendered by `components/queue-org-manage/StatusBadge.tsx` and acted on by `AdminApprovalActions.tsx` (approve via `queueOrganization/:id/approve`). Transitions are server-enforced and broadcast as `queue_org_approved` / `queue_org_updated` / `queue_org_deleted` events, which invalidate the `QueueOrganizations` tag (Section 8.2, C30).

11.3 Session states

The `auth` Redux slice exposes `{ auth, hydrated }`. States are: *anonymous* (no profile; routing redirects to `/login`), *hydrating* (profile being restored from storage before guards decide), *authenticated* (profile present, socket connected) and *expired* (a 401 clears the profile and returns the user to `/login`). The hydration gate prevents a flash of protected content before the guard decision (C14, C31).

11.4 Realtime connection state

Socket connectivity is mirrored into the zustand store and surfaced by `useQueueSocket` as `{ socketConnected, isLive }`. A `connect_error` whose message indicates an unauthorised or token failure ends the session rather than retrying indefinitely (C8, Section 8.4).

11.5 Known gaps, assumptions and limitations

The following behaviours are recorded so that reviewers can distinguish them from the secured paths above. Each maps to an open finding in Section 15.3.

- **Settings are display-only.** Profile save, push/email/2FA and account deletion are presented but not persisted; save attempts are refused with an explicit message rather than silently faked (C28, F4).
- **Check-in is not geofenced.** A modelled `checkinRadiusKm` value is not enforced as a physical gate; only the driver id and optional queue number are validated (C16, F7).
- **Auth roleId.** Login/OTP/register transmit `roleId: 11` although the API contract comment states no role should be sent (C3, F1).
- **Host-level controls.** TLS/HSTS, CSP and other response security headers are a hosting/server responsibility and are not asserted by the repository (C25, F5).
- **No token refresh.** Session expiry forces re-authentication; there is no silent refresh (C14, F6).

12. Validation, error handling and logging

12.1 Validation schemas

All user-entered data passes through Zod schemas in `src/schemas/queue.ts` before submission. The schemas bound lengths, restrict enumerations and coordinate ranges, and transform values through the sanitizers below, providing client-side parity with backend validation (C16).

Schema	Used by	Key rules
`loginSchema`	Login	Phone 10–20 chars, digits/`+` only
`registerSchema`	Register	Name 2–100 (sanitised); phone 12–20; email valid or empty
`setupOrgSchema`, `createQueueOrgSchema`	Setup organisation	Name ≤ 255; enum type; address required; lat/long bounded
`queueOrgProfileSchema`	Org profile form	Same as setup, with optional bounded coordinates
`dispatchSchema`	Dispatch modal	Optional shipper request id must be a UUID if present
`checkinSchema`	Check-in modal	Driver id UUID; optional positive integer queue number
`overrideSchema`	Override modal	Queue number ≥ 1; reason ≤ 500 (sanitised)
`createOrderSchema`	Create order modal	Quantity > 0 ≤ 10,000 quintals and ≤ vehicle capacity; non-negative cost; delivery date ≥ shipping date; bounded coordinates (C20)
`uuidSchema`	Shared	`z.string().uuid()` for all identifiers (C20)

12.2 Input sanitisation and output encoding

Text and email inputs are sanitised in the schema layer by `stripTags`, `stripCtrl`, `sanitizeText` and `sanitizeEmail`, which remove HTML tags and control characters, collapse whitespace and normalise email (`src/schemas/queue.ts:7–17`). Rendering relies on React's automatic output escaping; the codebase contains no `dangerouslySetInnerHTML`, `innerHTML` or `eval` usage (C17, C18). Together these close the stored/reflected XSS vector of Annex A#4.

12.3 Phone normalisation

`utils/phoneFormatter.ts` strips a leading `+251`, `251` or `0`, caps input at nine digits, renders the `XX–XX–XX–XX` mask and produces E.164 `+251…` for transmission. `isValidPhoneDigits` accepts exactly nine digits (C19).

12.4 Error handling

API errors are normalised by `getApiError` (`lib/api.ts`) and `parseError` (`utils/parseError.ts`) into a safe message that is shown as a toast via `sonner` (`<Toaster richColors />` in `main.tsx`). Raw

server stack traces, SQL or internal identifiers are never surfaced to the user. A 401 from any request routes through the interceptor to `clearAuth()` and a redirect to `/login`, unless the user is already on the login page (C14, C35).

12.5 Logging and build artefacts

The production bundle is built with source maps disabled (`vite.config.ts`), so client code is not shipped in readable form, and no token or credential is written to logs (C29, C35). Application monitoring, audit logging and log retention are server-side responsibilities (`NFR-DEPLOY`, C25); the client's only diagnostic surface is the transient toast described above.

13. Security implementation

13.1 Trust boundaries

The client is an untrusted rendering surface. Every privilege decision that matters is made by the backend service; the browser-side guards and role allow-lists exist for usability and defence in depth. Figure 2.1 (Section 2) shows the boundary between the SPA, the API and the Socket.IO channel, all of which authenticate with the same session cookie. This is the design's response to Annex A#8 (lack of authentication and authorisation) and Annex B tests 23–28.

13.2 Cross-site scripting (XSS)

All rendering flows through React's escaped JSX, and no unescaped-HTML sink is used (C18). User-supplied text is additionally stripped of tags and control characters at the schema boundary (C17). No user content is injected into URLs or attribute positions without encoding, addressing Annex A#4.

13.3 Authentication and session management

Authentication is phone + one-time code; the session is a backend-issued httpOnly cookie set at `verifyUserByOTP`. JavaScript never receives, stores or transmits a bearer token — the only persisted auth value is the non-sensitive profile, and the legacy token key is actively purged (C11, C12). Requests carry cookies via `credentials: "include"`; there is no `Authorization` header. A 401 or an unauthorised socket error ends the session immediately (C8, C14), and logout performs a best-effort server call before clearing local state (C15). This addresses Annex A#3 (improper session management) and Annex B tests 63–77.

13.4 Cross-site request forgery (CSRF)

Because the session is cookie-based, CSRF protection depends on the cookie's `SameSite` attribute and the backend's cross-origin policy, both of which are server-side settings (C25, F5); the client sends no bespoke CSRF token. This dependency is recorded rather than claimed as a client control (Annex A#5).

13.5 Transport security

The canonical origin is HTTPS (`index.html` canonical and Open Graph URLs, plus `public/sitemap.xml`), and the API base resolves to an absolute `/api` URL by `baseUrl.ts` (C21). There is no client-side use of `http://` resources and no mixed content (C22). The validity and renewal of the TLS certificate, and the presence of HSTS, are hosting obligations (C25, Annex B#14–18).

13.6 Information disclosure

`public/robots.txt` allows the public site and points only at `sitemap.xml`; it lists no sensitive paths (C26). The client bundle carries only `VITE_`-prefixed public variables and no secrets (C27), and production builds are shipped without source maps (C29). There are no shipped debug routes or developer panels beyond the intentional `/security` self-assessment page (C35).

13.7 Supply chain and dependency hygiene

Dependencies are declared with pinned lockfile resolution and no runtime CDN imports; the build is fully local (C33). Runtime dependencies are limited to the React/Redux/RTK Query, routing, forms,

validation, socket, i18n and chart-free UI stack listed in `package.json`. An unused `@tanstack/react-query` dependency remains in the manifest (F3).

13.8 Control summary

Each implementation decision above is carried as a numbered control C1–C36 in the register of Section 15.1, where it is mapped to the specific file and symbol that provides the evidence. Open items and server-side dependencies are listed in Section 15.3.

14. Testing strategy

14.1 Unit test suite

The suite runs under Vitest (`npm test` = `vitest run`); its include pattern is `src/**/*.{test,spec}.?(c|m)[jt]s?(x)` (`vite.config.ts`). There are 98 tests across seven suites in `src/__tests__/`, all passing at the submitted revision.

Suite	Focus
`insaStandard.test.ts`	Standard metadata; 5 focus areas and 35 requirements with unique ids and valid levels; Annex A vulnerabilities 1–8; Annex B tests 1–102 in 8 groups with no gaps or duplicates; 5 principles
`journeyStatus.test.ts`	Journey status vocabulary and label resolution
`ordersDataMapper.test.ts`	Mapping of order/shipper-request payloads to view models
`ordersLocation.test.ts`	Address trimming, route formatting, intra-city handling, distance calculation and offer-cost resolution
`phoneFormatter.test.ts`	Digit cleaning, display mask, E.164 round-trip and nine-digit validation
`queueMutations.test.ts`	Queue mutation reducers/helpers (dispatch, override, membership)
`securityChecklist.test.ts`	Checklist parsing, bounds, toggles and statistics

14.2 Static and build gates

Three gates guard a release (C32): the TypeScript project build (`npm run build` = `tsc -b && vite build`), the linter (`npm run lint` = `eslint . --ext ts,tsx --max-warnings 0`, with `tsconfig.app.json` enforcing `noUnusedLocals`, `noUnusedParameters` and `verbatimModuleSyntax`), and the unit suite. At the submitted revision `tsc` and the build pass and all 98 tests pass; the frontend has a pre-existing ESLint baseline of 185 errors and the backend 55, so `npm run lint` does not currently exit clean at HEAD even though no new violations are introduced by this work (open finding F2). `.oxlintrc.json` is present but `oxlint` is not wired into the `lint` script.

14.3 Test coverage scope and gaps

Coverage targets the pure logic where correctness is machine-checkable: standard data integrity, status mapping, payload mapping, formatting and checklist state. Browser-level flows (authentication, routing guards, realtime) are verified by review and manual testing rather than automated end-to-end tests, which are out of the submitted scope. This limitation is carried in Section 15.3 (F5, F8).

15. Control-to-code traceability

15.1 Control register (C1–C36)

Each control names a single verifiable statement, the INSA reference it derives from, and the file-and-symbol evidence in the submitted tree. Status is MET (implemented in the client), OPEN (deficiency or outstanding work) or BY DESIGN (deliberately not a client control).

ID	INSA ref	Control statement	Evidence (file / symbol)	Status
C1	2.1 RGA-C/D	Security requirements and their realisation are traceable from requirement to code.	`docs/insa/SRS.html`, `docs/insa/SDD.html`, this document §15.2	MET
C2	2.2 DES-B	The design is identified and controlled as an organisational asset.	Cover metadata `ASSET-DYN-QA-001`; Appendix B document control	MET
C3	2.5 OPM-D; A#8	Role is resolved from the authenticated account under least privilege; the client does not assert its own role.	`src/services/auth.service.ts` (sends `roleId: 11`) — deviation F1	OPEN
C4	B#40, B#64	Authentication is multi-stage; the session is issued only after code verification.	`components/auth/VerifyOtp.tsx`; `auth.service.ts → verifyOtp()`	MET
C5	B#74	The realtime channel authenticates with the same session credentials as HTTP.	`src/lib/socket.ts` (`withCredentials: true`, `auth` identity)	MET
C6	B#24, B#32	Protected routes fail closed; unauthenticated states redirect to login.	`components/auth/ProtectedRoute.tsx`	MET
C7	2.5 OPM-D; B#23	Privileged routes enforce a role allow-list.	`components/auth/RoleGuard.tsx` (`{11,12,3,6}`)	MET
C8	B#63, B#76	Sessions are revoked on logout and on an unauthorised realtime error; reconnection is bounded.	`AuthContext.tsx`; `src/lib/socket.ts:168-178`	MET
C9	B#24	Client authorisation is defence in depth; the server remains the authority.	Guards + `lib/api.ts` 401 handling; documented in §13.1	BY DESIGN
C10	2.5 OPM-D	Actions are scoped to the selected organisation; member administration is gated to role 11.	`MembersTable.tsx` (`canManage`); `queueSlice selectedOrgId`	MET
C11	B#68, B#9	The session token is never stored in or read by JavaScript; it lives in an httpOnly cookie.	`lib/auth.ts`; `lib/api.ts` (`withCredentials`)	MET
C12	B#9	A legacy token-bearing storage key is purged on load.	`lib/auth.ts → purgeLegacyAuth` (`queueadmin:auth`)	MET
C13	B#74	Cross-origin requests transmit cookies only, with credentials mode enabled.	`lib/redux/api/base.ts`; `lib/api.ts`	MET

ID	INSA ref	Control statement	Evidence (file / symbol)	Status
C 14	B#63, B#66	A 401 clears authentication and returns the user to login.	`lib/api.ts` interceptor; `base.ts` `baseQueryWithReauth`	MET
C 15	B#63	Logout revokes the server session (best effort) and clears local state.	`AuthContext.tsx` → `logout()` (`user/logout`)	MET
C 16	B#59	All form input is validated against bounded schemas before submission.	`src/schemas/queue.ts` (all schemas)	MET
C 17	A#4; B#43	Text input is sanitised of tags and control characters.	`src/schemas/queue.ts:7–17` (`sanitizeText`, `sanitizeEmail`)	MET
C 18	A#4; B#43	Output is encoded; no unescaped-HTML or script sinks are used.	React JSX escaping; no `dangerouslySetInnerHTML` / `eval` in tree	MET
C 19	B#59	Phone identifiers are normalised and length-validated.	`utils/phoneFormatter.ts`	MET
C 20	B#28	Identifiers are constrained to UUIDs; capacity and quantity bounds are enforced.	`schemas/queue.ts` (`uuidSchema`, `createOrderSchema`)	MET
C 21	B#14	All traffic uses HTTPS against an absolute API base.	`lib/baseUrl.ts`; `index.html` canonical `https://queue.dynamicsroute.tech/`	MET
C 22	B#15	No mixed content; assets are same-origin.	`index.html`; `vite.config.ts` (local build, no CDN)	MET
C 23	B#21	Realtime events are not trusted as state; they trigger debounced cache invalidation.	`src/lib/socket.ts` (100 ms org / 250 ms other)	MET
C 24	B#21, B#74	Subscriptions are refcounted and replayed on reconnect; identity matches backend fan-out.	`src/lib/socket.ts:156–161`; `extractCredentials()`	MET
C 25	2.5 OPM-N; B#16-18	TLS certificate validity, HSTS and security headers are maintained by the host.	Hosting configuration (outside repository) — F5	BY DESIGN
C 26	B#5	`robots.txt` and sitemap expose no sensitive paths.	`public/robots.txt`, `public/sitemap.xml`	MET
C 27	B#6	No secrets are shipped to the client; only `VITE_` public variables are consumed.	`.env.example`; `lib/baseUrl.ts`	MET
C 28	B#9	Client persistence is minimal and non-sensitive.	`lib/auth.ts`, `ThemeContext.tsx`, `i18n/index.ts` (§10.2)	MET
C 29	B#6	Production builds ship without source maps.	`vite.config.ts`	MET

ID	INSA ref	Control statement	Evidence (file / symbol)	Status
C 30	B#21	Business state is server-authoritative; the client cache converges via tag invalidation.	`lib/redux/api/base.ts` (9 tags)	MET
C 31	B#22	Route/guard decisions wait for auth hydration; no invalid state transition renders.	`authSlice` (`hydrated`); `ProtectedRoute`	MET
C 32	2.3 IMP-A; 2.5 OPM-I	Type-check, lint and unit tests act as release gates.	`package.json` scripts; §14.2 (baseline F2)	OPEN
C 33	B#90, B#91	Dependencies are locked and locally built; no runtime CDN code.	`package-lock.json`, `package.json`	MET
C 34	2.5 OPM-I, OPM-M	Environments are separated and change control applies; no secret is committed.	`.env.example`, build modes; `.gitignore`	MET
C 35	2.5 OPM-K; B#3, B#4	No debug surfaces ship; errors and logs reveal nothing sensitive.	`utils/parseError.ts`; toasts; server logging (F5)	MET
C 36	2.3 IMP-E; 2.5 OPM-B	Documentation and the INSA assessment record are maintained.	`docs/insa/*`; `pages/security/SecurityPage.tsx`	MET

15.2 Requirement-to-code traceability

Family	Design element	Primary code	Verification
`FR-AUTH`	Phone + OTP login and registration	`components/auth/*`; `services/auth.service.ts`	Manual review; `phoneFormatter.test.ts`
`FR-ROLE/ACCESS`	Authentication and role guards	`ProtectedRoute.tsx`; `RoleGuard.tsx`; `AppRoutes.tsx`	Manual review
`FR-ORG`	Setup, profile, approval, membership	`organization/SetupOrganization.tsx`; `queue-org-manage/*`	`queueMutations.test.ts`
`FR-QUEUE`	Board, dispatch, check-in, override, history	`pages/dashboard/QueueDashboardPage.tsx`; `queue/*`	`queueMutations.test.ts`
`FR-BID`	Bid listing, comparison, explicit acceptance	`orders/DriverBidsModal.tsx`; `orders/bids/*`	`ordersDataMapper.test.ts`
`FR-ORDER`	Orders list, mapping, location, creation	`pages/orders/*`; `orders/*`; `utils/formatters.ts`	`ordersDataMapper.test.ts`; `ordersLocation.test.ts`
`FR-REALTIME`	Socket lifecycle, subscriptions, cache invalidation	`lib/socket.ts`; `hooks/useQueueSocket.ts`	Manual review
`FR-REPORT`	Operator metrics	`pages/reports/ReportsPage.tsx`; `hooks/useReportsMetrics.ts`	Manual review

Family	Design element	Primary code	Verification
NFR-SEC	Validation, sanitisation, checklist, INSA data	schemas/queue.ts; securityChecklist.ts; data/insaStandard.ts	securityChecklist.test.ts; insaStandard.test.ts
NFR-DEPLOY	Build, tooling, PDF rendering	vite.config.ts; package.json; scripts/render-pdf.cjs	Build and gates (§14.2)

15.3 Open findings and residual items

Ref	Finding	Control	Disposition
F1	Login/OTP/register transmit roleId: 11, contradicting the API-contract comment and README.	C3	Documented as-built; contract drift to be resolved with the backend.
F2	Frontend ESLint baseline of 185 errors (backend 55); npm run lint does not exit clean at HEAD.	C32	No new violations introduced; baseline cleanup tracked.
F3	Unused @tanstack/react-query dependency in the manifest.	C33	Dependency to be removed in a hygiene pass.
F4	Settings (profile, notifications, 2FA, deletion) are display-only and not persisted.	C28	By design — refused explicitly rather than faked.
F5	CSP, HSTS and other response security headers, plus CSRF SameSite, are server-side and not asserted in-repo.	C25, C35	Hosting/server responsibility; to be evidenced separately.
F6	No token refresh; session expiry forces re-authentication.	C14	By design.
F7	Check-in is not geofenced; checkinRadiusKm is not enforced.	C16	By design — optional queue number with position estimate only.
F8	No automated end-to-end tests for auth, routing or realtime.	C32	Coverage gap; verified by review and manual testing.
F9	Social bonding features deferred.	C36	Deferred by scope decision; documented.
F10	pages/organizations/OrganizationsPage.tsx is legacy/unrouted.	C36	Housekeeping; retained pending removal decision.

16. Deployment and operations

16.1 Build and release

The release artefact is a static bundle produced by `npm run build` (`tsc -b && vite build`) on Node ≥ 20.19.0. Vite emits hashed, minified assets to `dist/`, splits vendor/redux/icon chunks (`vite.config.ts`) and disables source maps in production (C29). The INSA document set is rendered to PDF with the same sources as the master: `node scripts/render-pdf.cjs --all` renders every `docs/insa/*.html` and audits each PDF for page count and horizontal padding (C36).

16.2 Configuration

Runtime configuration is limited to public `VITE_` variables resolved at build time by `lib/baseUrl.ts`, in priority order: `VITE_API_BASE_URL` → `VITE_API_URL` → `/api`, normalised to end in `/api`. No secret is provided to the client build (C27). The dev server proxies `/api` and the Socket.IO upgrade path only when `VITE_API_BASE_URL` is set (C34).

16.3 Hosting and transport

The console is served as a single-page application under the canonical HTTPS origin `https://queue.dynamicsroute.tech/`. The host is responsible for TLS termination, certificate renewal, HSTS, security headers, the cookie's `Secure` / `HttpOnly` attributes and the SPA fallback route; these are the server-side dependencies recorded in C25 and finding F5.

16.4 Operations, monitoring and disclosure

Application monitoring, audit logging, rate limiting on authentication and e-mail, and incident response are backend responsibilities (`NFR-DEPLOY`, C25). On the client side, `public/robots.txt` and `public/sitemap.xml` expose only the public root (C26), and no operational endpoints or debug surfaces are shipped (C35).

16.5 Business continuity and change control

Backups, disaster recovery and continuity are owned by the hosting provider. Changes to the client follow the build gates of Section 14.2 and the environment separation of Section 16.2; the repository commits no secrets and ignores local environment files (C34). Significant changes are expected to pass an operational risk assessment before release (2.5 OPM-J), which is a process rather than a code control and is therefore recorded here rather than in the register.

Appendix A — file/module index

This index lists the principal modules of the submitted tree. Paths are relative to the `queadmin-frontend` repository root; the source tree contains roughly 228 tracked files under `src/`.

Path	Role
`src/main.tsx`	Entry point: StrictMode, Redux Provider, ThemeProvider, AuthProvider, Router, Toaster; imports i18n and CSS
`src/App.tsx`, `src/routes/AppRoutes.tsx`	Application shell and the route/guard table
`src/components/auth/*`	Auth step controller, login, OTP, register, shared fields, guards
`src/context/AuthContext.tsx`	Session lifecycle, socket connect/disconnect, logout
`src/services/auth.service.ts`	Login/OTP/register/logout HTTP calls
`src/lib/api.ts`, `src/lib/auth.ts`, `src/lib/baseUrl.ts`	Axios client, profile storage migration, API base resolution
`src/lib/socket.ts`, `src/hooks/useQueueSocket.ts`	Realtime connection, subscription refcount, cache invalidation
`src/lib/redux/api/*`	RTK Query base, tags, endpoints (auth, organisations, queue, shipper requests, vehicles), types
`src/lib/redux/slices/*`, `src/lib/redux/store.ts`	Auth and queue slices, store wiring
`src/store/queueAdminStore.ts`	Zustand store (selected org + socket connectivity) with persistence
`src/components/organization/SetupOrganization.tsx`	Organisation creation flow
`src/pages/queue-org-manage/*`, `src/components/queue-org-manage/*`	Organisation management: profile, approval, members
`src/pages/dashboard/QueueDashboardPage.tsx`, `src/components/queue/*`	Live queue board, dispatch, check-in, override, entry history, order creation
`src/pages/orders/*`, `src/components/orders/*`	Orders and shipper-request views, bids, mobile cards, mapping
`src/pages/reports/*`, `src/hooks/useReportsMetrics.ts`	Reports and metrics
`src/pages/security/*`, `src/components/security/*`	INSA assessment page and checklist
`src/pages/settings/*`, `src/contexts/ThemeContext.tsx`, `src/i18n/index.ts`	Display-only settings, theme and language persistence

Path	Role
`src/schemas/queue.ts`	Zod validation schemas and sanitizers
`src/types/queue.ts`	Domain types and enums
`src/utils/*`	Journey status, vehicle types, phone formatting, formatters, error parsing, checklist, constants
`src/data/insaStandard.ts`	Embedded INSA standard: focus areas, vulnerabilities, tests
`src/__tests__/*`	Seven Vitest suites (98 tests)
`scripts/render-pdf.cjs`	Headless-Chrome PDF renderer and layout auditor for the INSA documents
`public/*`	favicon, icon sprite, robots.txt, sitemap.xml
`index.html`, `vite.config.ts`, `tsconfig*.json`, `eslint.config.mjs`, `.oxlintrc.json`	HTML shell and build/tooling configuration

`src/pages/organizations/OrganizationsPage.tsx` is present but not routed and is not part of the shipped navigation (finding F10).

Appendix B — document control

B.1 Version history

Version	Date	Status	Author	Summary
1.0.0	15 October 2026	As-built	Marew Masresha, General Manager	Initial TDD/LLD for the Dynamics Queue Admin console (ASSET-DYN-QA-001), covering Sections 1–16 and Appendices A–B.

B.2 Approval

Role	Name	Signature	Date
Prepared by	Marew Masresha, General Manager	_____	15 October 2026
Reviewed by	Marew Masresha, General Manager	_____	15 October 2026
Approved by	Marew Masresha, General Manager	_____	15 October 2026

B.3 Reference documents

- INSA Secure Website Management Standard, Version 1.0, 2014 EC (with Annex A vulnerabilities and Annex B test list).
- `docs/insa/SRS.html` — Software Requirements Specification (requirement families).
- `docs/insa/SDD.html` — System & Architecture Design (Section 12 requirement map).
- `docs/insa/UserGuide.html` — User Documentation.
- `docs/insa/INSA_Security_Submission.html` — consolidated security submission.

B.4 Glossary

Term	Meaning
Console / SPA	The `queadmin-frontend` single-page application described here.
Queue organisation	A dispatch organisation (role 11) that manages a driver queue.
Dispatcher	Role 12 operator within a queue organisation.
Shipper request / order	A transport request and its derived journey/order records.
Bid	A driver's offer against a shipper request.
Control (C#)	A traceable security or quality statement in the register of Section 15.1.
Finding (F#)	An open or residual item in Section 15.3.
HTTP-only cookie	A session cookie not readable from JavaScript.

End of document

This TDD/LLD is the technical companion to the SRS, SDD, User Guide and Security Submission for the Dynamics Queue Admin console (`queadmin-frontend`), asset `ASSET-DYN-QA-001` , design version 1.0.0 (as-built).